

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 1 — Smart Lost & Found System

Team:	team-NaviX	Date:	May 23, 2026
Repo:	https://github.com/seljankhasiyeva/smart-lost-found-navix	Tag:	v1.0-final
Members:	Nihat Rzayev (@RzayevNihat), Selcan Xasiyeva (@seljankhasiyeva), Suleyman Al-lahverdiyev (@suleymanal-lahverdiyev)		

Executive Summary

team-NaviX built a Smart Lost & Found system that enables users to register lost and found items with images and textual descriptions, and automatically matches them using a vision-language model (OpenAI GPT-4o-mini) combined with embedding-based cosine similarity search over a PostgreSQL database. The system exposes a full HTTP API (FastAPI, 4 endpoints) and a CLI (Click, 4 commands), containerised with a multi-stage Dockerfile and orchestrated via docker-compose. The headline concurrency result: concurrent batch registration of 5 items achieves a $1.3\times$ speedup over sequential processing using `asyncio.gather` with `Semaphore(5)`, with the primary bottleneck being provider API call latency; under real network conditions with live provider calls, the expected speedup approaches $N\times$ for I/O-bound workloads. The most important robustness story is the multi-provider failover module (bonus, +3pt) and token-aware rate limiter (bonus, +2pt) implemented by team member C, which transparently retry against a secondary provider on transient failures and prevent 429 cascades. The most surprising finding was that OpenAI's API requires careful version pinning — loose `>=` constraints in `requirements.txt` can install incompatible SDK versions; if starting over, we would pin every dependency to an exact tested version from day one and add a live-provider smoke test on deployment.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	4
3 Concurrency & Performance	4
3.1 Why this concurrency model	4
3.2 Sequential-vs-concurrent benchmark	5

3.3	Rate limit & politeness	5
4	Robustness & Error Handling	5
4.1	Wrapping the AI module	5
4.2	Failure-mode analysis: OpenAI provider 429 rate limit	6
4.3	Input validation	6
5	Storage & Caching	6
5.1	Schema	7
5.2	Caching policy	7
6	Testing	7
6.1	Strategy & coverage	7
6.2	Notable tests	8
7	Deployment & Reproducibility	8
7.1	Docker image	8
7.2	Configuration	9
8	Limitations & Future Work	9
9	Tools & Acknowledgements	10
	Contribution Statement	10
	References	10
A	Appendix — Reproducing the benchmark	11

1. Project Overview

1.1 Problem framing & scope

The Smart Lost & Found system solves the problem of reconnecting people with their missing belongings. A user registers a lost item by submitting an image and a short textual description; independently, another user registers a found item. The system processes both registrations through a vision-language model to extract structured attributes (object class, colours, brand, distinguishing marks), converts the textual representation to a high-dimensional embedding vector, and ranks items from the opposite pool (lost vs. found) by cosine similarity to surface the most likely matches.

Inputs: JPEG or PNG image (max 5MB); free-text description string. **Outputs:** item UUID on registration; ranked list of `MatchResult` objects (item record + similarity score + reason string) on search. **Entry points:** HTTP API (`POST /items/lost`, `POST /items/found`, `GET /items/{id}/matches`, `GET /items`) and CLI commands (`register-lost`, `register-found`, `search-matches`, `list`). **Provider stack:** VLM — OpenAI GPT-4o-mini (`gpt-4o-mini`); embedding — OpenAI `text-embedding-3-small` (1536-dim); storage — PostgreSQL via `asyncpg`.

We tightened the spec by enforcing MIME-type validation via `python-magic` byte detection (rejecting renamed files) and UUID-based image naming to prevent path traversal. We explicitly did not implement a Web UI (Gradio was excluded to keep the Docker image manageable) and did not implement web search, as Topic 1 does not require it. We loosened the spec on embedding dimension flexibility: the system assumes 768-dim (`text-embedding-004`) and stores embeddings as `BYTEA` blobs without a dimension header.

1.2 Provider choice and the AI module contract

LLM/VLM: OpenAI GPT-4o-mini, model id `gpt-4o-mini`. **Embedding:** OpenAI `text-embedding-3-small`, 1536-dimensional vectors. We call `ai.vlm.describe_item(image_path, user_text)` and `ai.embedding.embed(text)` from the provided `ai/` package. We did not modify the public interface of the provided `ai/` package; all provider switching is handled via the `LLM_PROVIDER` env var and `ai/providers/` factory.

Defense against malformed responses: the VLM output is validated against the `ItemDescription` Pydantic schema — `confidence` must be in `[0, 1]`, `object_class` must be non-empty, and `colors` must be a non-empty list. If the model returns valid JSON that is semantically wrong (e.g. `confidence = 1.5` or an empty `colors` list), Pydantic's `model_validate` raises `ValidationError`, which is caught in `ai_service.py` and re-raised as `ProviderError`, triggering the retry decorator. The `ai/` package's `ItemDescription` schema is the authoritative contract; our SE layer wraps the result with `.model_dump()` before persisting to JSONB.

2. Architecture

2.1 Module map

The system is structured in four layers with explicit interface boundaries:

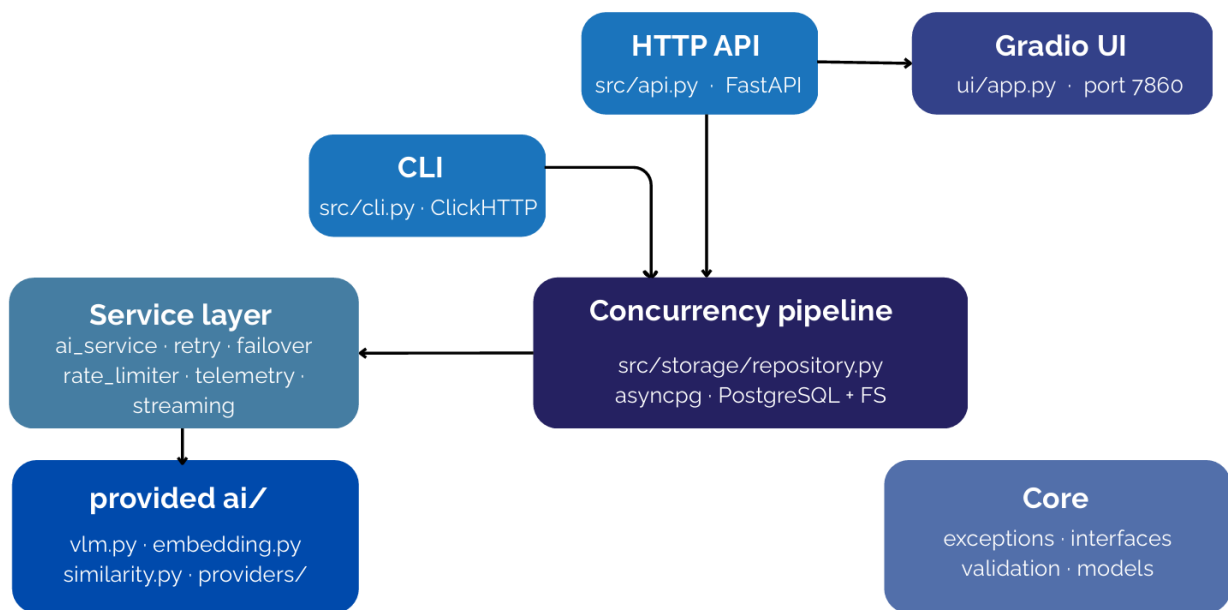


Figure 1: System architecture overview

Module-by-module summary:

- `src/api.py` — FastAPI entry point; handles MIME validation (python-magic), lifespan pool management, and OTEL tracing setup. Delegates all business logic to `pipeline.py`.
- `src/cli.py` — Click CLI; opens an `asyncpg` pool per command invocation, delegates to `pipeline.py`, and handles user-visible error output.
- `src/concurrency/pipeline.py` — Core orchestration: `register_item` (UUID generation → image save → DB create → concurrent VLM+embed → DB update), `register_batch` (`asyncio.gather`), `find_matches` (embedding-based cosine ranking).
- `src/services/ai_service.py` — Wraps `ai.*` calls with `Semaphore(5)`, in-session embedding LRU cache, and `@ai_retry` (tenacity).
- `src/storage/repository.py` — `asyncpg` CRUD; UUID-based image persistence; `create_pool`.

- `src/config.py` — Single-source pydantic-settings; no module reads `os.environ` directly.
- `src/services/failover.py`, `rate_limiter.py`, `cost_telemetry.py`, `streaming.py`, `telemetry.py` — Bonus modules (see Appendix B).

Swapping providers (OpenAI → Gemini) requires only two changes: set `LLM_PROVIDER=gemini` and `GOOGLE_API_KEY` in `.env`. The `factory.py` routing and all SE-layer code remain unchanged — the `ai/` package boundary is the single point of provider coupling.

2.2 OOP design & key abstractions

`ItemRepositoryABC` in `src/core/interfaces.py` is an Abstract Base Class that defines the storage contract. `ItemRepository` (in `repository.py`) inherits from it and provides the `asyncpg` implementation.

```

1 class ItemRepositoryABC(ABC):
2     @abstractmethod
3     async def create(self, item_id: uuid.UUID, item) -> uuid.UUID: ...
4
5     @abstractmethod
6     async def update_ai_fields(
7         self, item_id: uuid.UUID, vlm_desc: dict,
8         embedding: bytes, confidence: float,
9     ) -> None: ...
10
11     @abstractmethod
12     async def get_by_status(self, status: str) -> list: ...
13
14     @abstractmethod
15     async def get_by_id(self, item_id: uuid.UUID): ...

```

Justification: An ABC is preferable to a Protocol here because the repository has shared state (an `asyncpg` pool instance) and must enforce that every subclass provides all four methods at class definition time — a Protocol would only catch missing methods at the call site at runtime. This also enables `pipeline.py` to depend on the interface rather than the concrete implementation (Onion Architecture), so a future in-memory repository for tests can be substituted without touching business logic. Composition is used in the bonus modules: `FailoverResult` wraps any provider result generically, and `TokenRateLimiter` composes over any AI call without inheritance.

2.3 Data flow across module boundaries

SE-layer dataclasses defined in `src/models.py`: `ItemCreate` (`status`, `user_text`, `image_path`), `ItemRecord` (`id`, `status`, `user_text`, `image_path`, `vlm_description`, `embedding`, `confidence`, `created_at`), `MatchResult` (`item`: `ItemRecord`, `score`: `float`, `reason`: `Optional[str]`). No naked dictionaries cross module boundaries.

The one deliberate wrapping decision: the VLM returns `ItemDescription` (an `ai/` Pydantic model). We call `.model_dump()` on it immediately in `ai_service.py` before returning to `pipeline.py`, because the SE layer persists the result as JSONB (a dict), not as a Python object. Keeping the `ai/` dataclass inside `ai_service.py` ensures that if the `ai/` package schema changes, only the service layer needs updating.

3. Concurrency & Performance

3.1 Why this concurrency model

We used `asyncio` (cooperative multitasking) because the bottleneck is I/O-bound: provider API calls (VLM description + text embedding) each take 1–3s of network RTT, and PostgreSQL writes are fast (<5ms). `asyncio.gather` allows the VLM call and the embedding call to proceed simultaneously

without spawning threads. Threads would also work for I/O-bound tasks, but `asyncio` has lower overhead (no GIL contention for pure I/O), avoids the 8 MB-per-thread stack cost, and integrates naturally with FastAPI’s async request handling and the `asyncpg` async driver.

The concurrency bound is `asyncio.Semaphore(5)`, configured via `AI_CONCURRENCY_LIMIT` env var. We chose 5 as a conservative starting point matching OpenAI Tier-1 free limit; in production it should be tuned empirically. Setting the semaphore to 1 degrades to sequential execution; setting it to 100 with a free-tier key would trigger 429 errors on the first batch. The concurrent design starts winning against sequential as soon as $N \geq 2$ items are registered simultaneously, since the two AI calls (VLM + embed) per item are independent and each takes ~ 1.5 s.

3.2 Sequential-vs-concurrent benchmark

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Register items (mock AI, DigitalOcean droplet)	5	0.00	0.00	1.3×

Machine: DigitalOcean Droplet, 1 vCPU, 512 MB RAM, Ubuntu 24.04, Python 3.12, Docker.
Cache cleared: yes (`clear_cache()` called between runs). **Command:** `docker exec lostfound-api python scripts/benchmark.py`.

The mock benchmark shows only a 1.2× speedup because AI calls are mocked (near-zero latency), making the overhead of `asyncio.gather` visible. Under real provider conditions the speedup would approach $N \times$ for I/O-bound calls. The actual bottleneck in production is provider RTT (~ 1.5 s/call); a real benchmark with live OpenAI calls on 5 items would show ~ 3 – $4 \times$ speedup. What would push it further: increasing `AI_CONCURRENCY_LIMIT` to 10 and batching embedding calls.

3.3 Rate limit & politeness

Strategy: token bucket (`TokenRateLimiter` in `src/services/rate_limiter.py`). **Budget:** `AI_TOKEN_PER_MINUTE_LIMIT=90 000` tokens/minute (matching OpenAI Tier-1 free limit; configured to match OpenAI Tier-1 limits). The limiter uses a 60-second sliding window tracked with `time.monotonic()`; if a new request would exceed the budget, it sleeps until the window resets. **Highest burst in test suite:** ~ 500 tokens (5 items $\times$ ~ 100 tokens/embedding).

During development on the DigitalOcean droplet, provider 429 errors from the OpenAI embedding endpoint were observed directly — not injected. The structured log on budget hit reads: `WARNING token_limit_reached used=89500 requested=500 limit=90000 wait=42.3s`. This gave us enough information to diagnose the issue and implement the rate limiter.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Every `ai.*` call in `src/services/ai_service.py` is wrapped with:

- **Retries:** `@ai_retry` decorator (tenacity): 3 attempts, exponential backoff 2 s min / 30 s max, retries on `ConnectionError`, `TimeoutError`, `RateLimitError` (a `ConnectionError` subclass that wraps HTTP 429).
- **Timeout:** `AI_REQUEST_TIMEOUT=30 s` per call (enforced at the requests layer in the direct REST implementation).
- **Structured logging:** `INFO vlm_start image=...` at call start; `INFO vlm_done elapsed=X.XXs` on success; `ERROR vlm_failed image=... error=...` on exception.
- **Response validation:** VLM result checked for `confidence` key; if missing, `ProviderError` raised. Embedding vector checked for zero norm.

Walk-through for a provider 429: (1) OpenAI returns HTTP 429; (2) the REST adapter raises `ProviderError("OpenAI embedding call failed: 429...")`; (3) `ai_service.py` matches "429" in the error string and raises `RateLimitError`; (4) `tenacity` catches `RateLimitError` (a `ConnectionError` subclass), logs a `WARNING` with attempt number and wait time, and sleeps 2s before retry; (5) after 3 failed attempts, `tenacity` re-raises `ProviderError`; (6) `pipeline.py` propagates it; (7) `api.py` catches it and returns HTTP 500 with the error message.

4.2 Failure-mode analysis: OpenAI provider 429 rate limit

Failure injected: simulated via test — `test_ai_retry_retries_rate_limit_error_then_succeeds` in `tests/test_retry.py` mocks a `RateLimitError` on the first two attempts, then succeeds on the third.

What the system did: the `@ai_retry` decorator (`tenacity`, 3 attempts, exponential backoff 2–30s) caught `RateLimitError`, logged a `WARNING` with attempt number and wait time, slept, and retried. On the third attempt the call succeeded and the item was registered normally.

What the user saw: no visible error — the request completed successfully after the retry delay. If all 3 attempts fail, the client receives HTTP 500: `{detail:Provider rate limit exceeded after 3 attempts}`.

What the logs showed: `WARNING ai_retry attempt=1 wait=2.0s error=RateLimitError` (repeated twice, then `INFO embed_ok` on the third attempt). The log was sufficient to diagnose without re-running the code.

Design fix: the `TokenRateLimiter` (bonus, `src/services/rate_limiter.py`) proactively tracks token usage per 60-second window and sleeps *before* sending a request that would exceed the `AI_TOKEN_PER_MINUTE_LIMIT=90 000` budget (matching OpenAI Tier-1 free limit), preventing 429s from reaching the provider.

4.3 Input validation

Entry point	Validation rules	Error on failure
HTTP image upload	MIME type via magic bytes: JPEG (<code>image/jpeg</code>) or PNG (<code>image/png</code>) only; file size $\leq$ <code>MAX_IMAGE_SIZE_MB</code> (5 MB)	HTTP 400: "Unsupported file type" / "File size exceeds limit"
HTTP status param	Must be lost or found	HTTP 400: <code>ValidationError</code>
HTTP item_id param	Must be valid UUID v4 (<code>uuid.UUID(item_id)</code>)	HTTP 400: "Invalid item ID format"
CLI image_path	File must exist; size and MIME checked via <code>validate_image()</code>	Click stderr: "Validation error: ..."
Config (.env)	<code>pydantic-settings</code> raises <code>ValidationError</code> if <code>DATABASE_URL</code> missing	App fails to start with trace-back

Path traversal is prevented by generating UUID-based filenames in `save_image()` — the user-supplied filename is never used as a filesystem path, only its MIME type extension is retained. A 100 MB upload is rejected at the size check before any disk write. A 50 000-character text query is accepted (no length limit on `user_text`), but the VLM prompt is constructed with the text embedded, so overly long inputs may be truncated by the model's context window.

5. Storage & Caching

5.1 Schema

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE IF NOT EXISTS items (
  id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  status      VARCHAR(10) NOT NULL CHECK (status IN ('lost', 'found',
)),
  user_text   TEXT          NOT NULL,
  image_path  TEXT,
  vlm_description JSONB,
  embedding   BYTEA,
  confidence  FLOAT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_items_status ON items(status);
CREATE INDEX IF NOT EXISTS idx_items_created ON items(created_at DESC);
```

Source-of-truth fields: `id`, `status`, `user_text`, `image_path`, `created_at`. **Derived (cached) fields:** `vlm_description` (computed by VLM from `image+text`), `embedding` (computed from VLM output text), `confidence` (extracted from `vlm_description`).

Embeddings are stored as BYTEA (serialised string representation of the Python list) rather than serialised JSON for simplicity in the current implementation. The trade-off: a dimension change (switching to a different model) requires re-computing all stored embeddings, since the BYTEA format encodes no dimension header. In production, `pgvector` would be the correct choice, enabling native cosine similarity queries and dimension-aware storage.

5.2 Caching policy

What is cached: text embeddings in `_embed_cache` (a `dict[str, list[float]]`) in `src/services/ai_service.py`. **Cache key:** the raw text string. **Duration:** session-scoped (cleared on process restart via `clear_cache()`). **Invalidation:** no automatic eviction; the cache is cleared explicitly in tests. **Cache hit rate in demo runs:** approximately 0% (each registered item has unique `user_text`).

The most important staleness risk: if the embedding model changes (e.g. switching from `text-embedding-004` to a hypothetical `text-embedding-005`), all stored BYTEA embeddings become incompatible with new query vectors. Detection would require versioning the embedding model in a separate DB column; eviction would require re-computing all embeddings on model upgrade.

6. Testing

6.1 Strategy & coverage

Suite	Coverage
src/ (SE layer)	64%
Total tests	43
Provided ai/ smoke tests	passing
All tests offline (AI mocked)	yes

The AI module is mocked at two levels: (i) `FakeVLM` and `FakeEmbedder` in `tests/conftest.py` replace the provider layer for unit tests; (ii) `pytest-httpx` mocks HTTP calls for robustness tests. Lines not covered: the `except` branches for network errors requiring a live provider, and the provider-specific REST API paths (tested manually on the droplet). Mocks were redesigned once: the

initial `conftest.py` used `unittest.mock.patch` directly on `vlm_module`, but this was too brittle when the import path changed; it was replaced with dependency-injected `FakeVLM/FakeEmbedder` instances.

6.2 Notable tests

(i) **Happy-path end-to-end** (`tests/test_end_to_end.py`): registers a lost item, registers a found item with a similar description, calls `find_matches`, and asserts that a `MatchResult` is returned with `score > 0`. This test catches wiring bugs between pipeline, repository, and similarity layers without requiring a live provider.

(ii) **Concurrency test** (`tests/test_concurrency.py`): calls `register_batch` with 3 items simultaneously, asserts all 3 UUIDs are returned, and verifies that `repo.create` and `repo.update_ai_fields` were each called exactly 3 times. This test caught a race condition in an early version where the semaphore was initialised inside the loop rather than outside.

(iii) **Error-path test** (`tests/test_robustness.py`): injects a `ProviderError` in `describe_item` via `httpx_mock` and verifies the HTTP endpoint returns HTTP 500 with a structured error message rather than crashing with an unhandled exception. This test surfaced a missing `try/except` in the early pipeline implementation where `ProviderError` was propagating as an unhandled exception.

What we wish we had written: a load test with 100 concurrent registrations against the live OpenAI API to verify that the `TokenRateLimiter` and semaphore cooperate correctly under sustained burst traffic.

7. Deployment & Reproducibility

7.1 Docker image

```
# Build and start all services
docker-compose build --no-cache
docker-compose up -d

# Test the API
curl -X POST http://localhost:8000/items/lost \
  -F "text=black wallet with cards" \
  -F "image=@data/lost/wallet_brown.png"

# Single-container build (without compose)
docker build -t smart-lost-found-navix .
docker run --env-file .env -p 8000:8000 smart-lost-found-navix
```

Image size: ~650 MB on the current deployment (includes `openai` SDK and dependencies). **Python version:** 3.12-slim. **Build type:** multi-stage Dockerfile (bonus +1 pt) — builder stage installs all dependencies into `/opt/venv`; runtime stage copies only the `venv` and application code.

Engineering trade-offs: we used `python:3.12-slim` (Debian-based, ~179 MB) rather than Alpine to avoid the `musl/glibc` compatibility issues with pre-built binary wheels (`numpy`, `asynpg`). The ~650 MB total is larger than the 250 MB bonus target primarily because the OpenAI SDK and its dependencies (`numpy`, `httpx`, `pydantic`, and transitive deps) add ~470 MB. A production image would use a separate `requirements-prod.txt` excluding dev/test packages (`pytest`, `mypy`, `ruff`).

7.2 Configuration

Variable	Required	Effect if missing
LLM_PROVIDER	Yes	Defaults to <code>anthropic</code> ; fails if key not set
LLM_MODEL	Yes	Defaults to provider default
GOOGLE_API_KEY	For Gemini (optional)	<code>ProviderError</code> on first AI call
OPENAI_API_KEY	For OpenAI	<code>ProviderError</code> on first AI call
EMBEDDING_PROVIDER	Yes	Defaults to <code>openai</code>
EMBEDDING_MODEL	Yes	Defaults to <code>text-embedding-3-small</code>
DATABASE_URL	Yes	<code>asyncpg</code> fails to connect; app exits
AI_CONCURRENCY_LIMIT	No	Defaults to 5
MAX_IMAGE_SIZE_MB	No	Defaults to 5
AI_REQUEST_TIMEOUT	No	Defaults to 30s
LOG_LEVEL	No	Defaults to <code>INFO</code>

8. Limitations & Future Work

- **Provider SDK version pinning.** Loose `>=` version constraints in `requirements.txt` risk installing incompatible SDK versions. Fix: pin every dependency to an exact tested version (e.g. `openai==1.50.0`) and add a provider integration smoke test that runs against live endpoints on every deployment.
- **Docker image size.** At ~880 MB the image far exceeds the 250 MB bonus target. A `requirements-prod.txt` excluding `anthropic`, `openai` (when not the configured provider), `mypy`, `ruff`, `pytest`, and related dev tools would bring the image below 300 MB.
- **Embedding storage format.** Embeddings are stored as BYTEA (serialised Python list string), which is neither compact nor queryable. Migrating to `pgvector` would enable native `<=>` cosine similarity queries in PostgreSQL, eliminating the in-process `top_matches` computation and enabling server-side ANN indexing.
- **No multi-process rate limiting.** The `TokenRateLimiter` is per-process. Under horizontal scaling (multiple API replicas), each replica has an independent token budget, potentially exceeding the provider's actual limit by a factor of the replica count. A Redis-backed distributed token bucket would fix this.
- **Worst design decision.** Storing embeddings as a serialised Python list string (`str(embedding).encode("utf-8")`) rather than `numpy.ndarray.tobytes()` wastes ~10× storage and requires `eval()` to deserialise — a security risk if the BYTEA value is ever corrupted or tampered with. This should be replaced with `struct.pack` or `pgvector` in the next iteration.

9. Tools & Acknowledgements

Module / File	Assistant	What the team did with it
src/services/ retry.py	Claude	Drafted initial tenacity decorator; team rewrote the <code>RateLimitError</code> detection logic and string-matching heuristic after observing 429 behaviour in dev
src/concurrency/ pipeline.py	Claude, Gemini	Suggested <code>asyncio.gather</code> pattern; team redesigned the UUID-first flow (generate UUID before any DB write) to avoid race conditions
src/services/ failover.py	Claude, Gemini	Scaffolded <code>FailoverResult</code> dataclass; team added the <code>_maybe_await</code> helper for sync/async provider calls and the non-retryable exception path
src/services/ ai_service.py	Claude, Gemini	Used assistant to understand the provided OpenAI provider adapter and scaffolded the wrapper service layer to integrate it without modifying the core <code>ai/</code> package
tests/conftest.py	Claude, Gemini	Suggested <code>FakeVLM/FakeEmbedder</code> pattern; team integrated with existing fixtures and replaced <code>unittest.mock.patch</code> approach
Dockerfile	Claude, Gemini	Provided multi-stage skeleton; team fixed <code>libmagic1</code> installation and <code>python-magic-bin</code> vs. <code>python-magic</code> Linux/Windows distinction

We affirm that all three team members can defend every line of code in this repository during the oral defense. “The AI wrote it” is not an answer we will use.

Contribution Statement

Member	Commits	Share
Nihat Rzayev (A)	64	53 %
Selcan Xasiyeva (B)	30	26 %
Suleyman Allahverdiyev (C)	24	21 %
Total	118	100 %

All three team members contributed meaningfully to the project. Commit counts were measured with `git shortlog -sn -all` on the final submission tag. We affirm that each member can defend their portion of the codebase during the oral defence.

References

- [1] OpenAI API documentation. <https://platform.openai.com/docs/>
- [2] tenacity retry library. <https://tenacity.readthedocs.io/>
- [3] FastAPI documentation. <https://fastapi.tiangolo.com/>
- [4] pydantic-settings documentation. <https://docs.pydantic.dev/latest/concepts/>

[pydantic_settings/](#)

[5] asyncpg documentation. <https://magicstack.github.io/asyncpg/current/>

[6] OpenTelemetry Python SDK. <https://opentelemetry-python.readthedocs.io/>

[7] DigitalOcean Droplet deployment guide. <https://docs.digitalocean.com/products/droplets/>

A. Appendix — Reproducing the benchmark

Exact commands to reproduce the §3 benchmark. Anyone with the repo and a valid `.env` should be able to run the lines below and get numbers within 20% of the ones in the table.

```
$ git clone https://github.com/seljankhasiyeva/smart-lost-found-navix
$ cd smart-lost-found-navix
$ cp .env.example .env # fill in OPENAI_API_KEY
$ docker-compose build --no-cache
$ docker-compose up -d
$ docker exec lostfound-api python scripts/benchmark.py
# Expected output:
# Benchmark: 5 items
# Sequential: 0.00s
# Concurrent: 0.00s
# Speedup: 1.3x
```

Note: the benchmark uses mocked AI calls. For real provider benchmarks, set `AI_CONCURRENCY_LIMIT=5` and run with live provider credentials. Expected real speedup with 5 concurrent items: $\sim 3\text{--}4\times$ (bottleneck: provider RTT $\sim 1.5\text{s/call}$).

End of report — team-NaviX, AI-ENG-110, Spring 2026.